

Lambda function – employee registration code:

1. IMPORTS (TOOLS USED)

```
import json
import boto3
from urllib.parse import unquote_plus
```

What they do:

- json → logging + debugging output
- boto3 → AWS SDK (Rekognition + DynamoDB)
- unquote_plus → decodes S3 object keys (important!)

Why unquote_plus?

- S3 file names may contain:

John%20Doe.jpg

- This converts it to:

John Doe.jpg

✓ Very important for correct processing

2. CONFIGURATION (SYSTEM SETTINGS)

```
REGION = 'eu-west-1'
BUCKET_NAME = 'samuel-employee-images'
TABLE_NAME = 'employee'
COLLECTION_ID = 'employees'
```

Meaning:

- REGION → AWS region (Ireland)
- BUCKET_NAME → where employee images are stored
- TABLE_NAME → DynamoDB table
- COLLECTION_ID → Rekognition face database

This is our **employee registration environment**

3. AWS CONNECTIONS

```
rekognition = boto3.client('rekognition', region_name=REGION)
dynamodb = boto3.resource('dynamodb', region_name=REGION)
table = dynamodb.Table(TABLE_NAME)
```

What's happening:

- Connect to Rekognition → for face indexing
- Connect to DynamoDB → for storing user data

4. MAIN FUNCTION (ENTRY POINT)

```
def lambda_handler(event, context):
```

Triggered by:

- **S3 upload event**

✓ When an employee image is uploaded → this function runs automatically

5. LOG INPUT EVENT

```
print("EVENT:", json.dumps(event))
```

Used for:

- Debugging
- Viewing event structure in CloudWatch logs

6. LOOP THROUGH RECORDS

```
for record in event['Records']:
```

S3 can send:

- One file
- Multiple files

✓ This ensures all are processed

7. GET BUCKET & FILE KEY

```
bucket = BUCKET_NAME  
key = unquote_plus(record['s3']['object']['key'])
```

What's happening:

- bucket → fixed (our employee bucket)
- key → filename from S3

Example:

employees/Larry_Page.jpg

DEBUG LOG

```
print(f"Processing: {key}")
```

Helps track which file is being processed

8. REKOGNITION (INDEX FACE)

```
response = rekognition.index_faces(
```

This is the **core registration step**

What `index_faces` does:

- Detects face in image
- Stores it in **Rekognition Collection**
- Generates a unique **FaceId**

PARAMETERS:

```
CollectionId=COLLECTION_ID
```

Our employee face database

```
ExternalImageId=key
```

Links face to image name

✓ Useful for tracing back later

MaxFaces=1

Only register one face per image

QualityFilter='AUTO'

Ensures:

- Good quality face detection
- Avoids blurry/invalid images

9. CHECK RESPONSE

```
face_records = response.get('FaceRecords', [])
```

Extract detected faces

NO FACE FOUND

```
if not face_records:  
    print("No face detected")  
    continue
```

If image has no face:

- Skip it
- Move to next

✓ Prevents system errors

10. GET FACE ID

```
face_id = face_records[0]['Face']['FaceId']
```

This is:

- Unique identifier for each person

✓ Used later in authentication

DEBUG

```
print("Face ID:", face_id)
```

11. SAVE TO DYNAMODB

```
table.put_item(  
    Item={  
        'rekognitionid': face_id,  
        'ImageKey': key,  
        'FirstName': 'Auto',  
        'LastName': 'Registered'  
    }  
)
```

What's happening:

We store:

Field	Meaning
rekognitionid	FaceId from Rekognition
ImageKey	File name
FirstName	Placeholder
LastName	Placeholder

This creates the **mapping**:

FaceId → Person Info

DEBUG

```
print("Saved to DynamoDB")
```

12. SUCCESS RESPONSE

```
return {  
    'statusCode': 200,
```

```
    'body': json.dumps("Done")
}
```

Confirms Lambda executed successfully

13. ERROR HANDLING

```
except Exception as e:
    print("ERROR:", str(e))
    raise e
```

If something fails:

- Logs error
- Throws exception (important for debugging)

FULL FLOW (REGISTRATION PIPELINE)

```
Upload Image → S3
      ↓
Lambda Triggered
      ↓
Rekognition index_faces
      ↓
Generate FaceId
      ↓
Store in DynamoDB
```

HOW THIS CONNECTS TO AUTHENTICATION

Registration file:

Creates:

Face → FaceId → DynamoDB record

Authentication file:

Uses:

Image → Match Face → Get FaceId → Lookup DynamoDB

SUMMARY

This file is responsible for:

- Detecting employee faces
- Registering them in Rekognition
- Storing identity mapping in DynamoDB
- Automating onboarding via S3 triggers

“This Lambda function is triggered by S3 upload events and is responsible for employee registration. It indexes faces into an Amazon Rekognition collection, extracts the generated FaceId, and stores the mapping between the FaceId and user metadata in DynamoDB, enabling future authentication workflows.”